Saturday, July 11, 2026 8:04 PM

Description

AWS Automation with Lambda & Boto3

Submission Requirements (applies to all assignments):

- **Push the Python code to GitHub and share the repository link.**
- **Provide a brief document explaining the steps you followed.**
- **Include screenshots of:**
- **IAM Role**
- **Lambda Configuration**
- **Test Invocation/Output**
- **CloudWatch Logs**
- **Final Result**

Your submission will be evaluated on:

- **Correctness of the solution**
- **Successful end-to-end implementation**
- **Clarity and organization of the documentation**
- **Assessment Weightage: This assignment carries 50 marks and is an important component of your Final Certification. Ensure your solution is complete, well-documented, and thoroughly tested before submission.**

Note: You only need to complete any 4 assignments for grading. However, practicing all the assignments is highly recommended, as these automation patterns are commonly used in real-world DevOps roles and technical interviews.

General notes for all assignments:

- **Use Python 3.12 or later as the Lambda runtime (Python 3.7/3.8 runtimes are deprecated).**
- **Use Amazon EventBridge for scheduling and event rules (the service formerly called CloudWatch Events).**
- **Use t3.micro for any EC2 instances (free-tier eligible in most regions).**
- **For IAM, write least-privilege inline policies wherever possible instead of attaching *FullAccess managed policies. This is what interviewers and real environments expect.**
- **Always print/log affected resource IDs so results are visible in CloudWatch Logs.**

Important : Cost Management Tips

If you clean up your resources after testing, these assignments should cost little to nothing.

Keep these points in mind:

- **Cost Explorer API (Assignment 4): Approximately ₹1 per API call. Test only a few times and do not schedule it.**
- **EC2: Stop or terminate instances immediately after capturing your screenshots.**
- **Snapshots & S3 Buckets: Delete them once your testing is complete.**

- Elastic IPs: Release any unused Elastic IPs.

Best Practices

- Use one AWS Region throughout (recommended: us-east-1).
- Complete the assignment and clean up resources in the same sitting.
- Set up a $1 AWS Budget Alert before you begin.
- Before logging out, ensure there are no unused EC2 instances, S3 buckets, or snapshots.

⚠ Most common mistake: Forgetting to terminate an EC2 instance, which can lead to unexpected AWS charges.

1.Automated S3 Bucket Cleanup (Objects Older Than 30 Days)

Objective: Automate deletion of stale objects in an S3 bucket.
Task: Delete files older than 30 days in a specific bucket.
Instructions:

1. S3 Setup: Create a bucket and upload several files. (Since you can't easily create "old" objects, temporarily lower the age threshold to minutes for testing — then set it back to 30 days in the final code.)
2. Lambda IAM Role: Inline policy with s3:ListBucket and s3:DeleteObject scoped to your bucket.
3. Lambda Function (Python 3.12+, Boto3):
4. List objects in the bucket (use the paginator — never assume one page of results).
5. Compare each object's LastModified (timezone-aware) with the current UTC time.
6. Delete objects older than 30 days.
7. Print the names of deleted objects.
8. Testing: Manually trigger and confirm only newer files remain.
9. Discussion point (include in your documentation): In production, S3 Lifecycle Rules handle this natively with zero code. Explain in 2–3 lines when you'd use Lambda instead (e.g., conditional logic, naming patterns, cross-service actions).

2.Automated EBS Snapshot Creation and Cleanup

Objective: Automate EBS volume backups and delete snapshots older than a retention period.
Instructions:

10. EBS Setup: Identify or create an EBS volume; note the volume ID.
11. Lambda IAM Role: Inline policy with ec2:CreateSnapshot, ec2:DescribeSnapshots, ec2:DeleteSnapshot, ec2:CreateTags.
12. Lambda Function (Boto3):
13. Create a snapshot of the specified volume and tag it (e.g., CreatedBy=Lambda-Backup).
14. List snapshots with that tag (owned by your account) and delete those older than 30 days.
15. Print IDs of created and deleted snapshots.
16. EventBridge: Schedule the function weekly.
17. Testing: Trigger manually; confirm snapshot creation and cleanup in the EC2 console.
18. Discussion point: AWS Data Lifecycle Manager (DLM) does this natively. Note in your documentation when Lambda is still the better choice (custom retention logic, cross-account copies, notifications).

3. Auto-Tagging EC2 Instances on Launch

Objective: Automatically tag newly launched EC2 instances for resource tracking, ownership, and cost allocation.
Instructions:

19. Lambda IAM Role: Inline policy with ec2:CreateTags and ec2:DescribeInstances.
20. Lambda Function (Boto3):
21. Extract the instance ID from the EventBridge event (detail.instance-id).
22. Tag the instance with LaunchDate=<current date> and a custom tag (e.g., Owner or Environment).
23. Print a confirmation message.
24. EventBridge Rule: Create a rule matching event pattern — source aws.ec2, detail-type EC2 Instance State-change Notification, state running — with the Lambda as target.
25. Testing: Launch a new instance; after a short delay, confirm the tags appear.
26. Bonus: Extract the launching IAM user from CloudTrail events and add an Owner tag automatically — this is a popular interview scenario.

4.Daily AWS Cost Alert Using Cost Explorer API and SNS

Objective: Build an automated alert when AWS spend exceeds a threshold.
Note: The old CloudWatch "Billing" metric is legacy — it only exists in us-east-1 and must be manually enabled. The modern, interview-relevant approach uses the Cost Explorer API (ce:GetCostAndUsage).
Instructions:

27. SNS Setup: Create a topic and subscribe your email (confirm the subscription email).
28. Lambda IAM Role: Inline policy with ce:GetCostAndUsage and sns:Publish (scoped to your topic).
29. Lambda Function (Boto3):
30. Initialize ce and sns clients.
31. Query month-to-date UnblendedCost with get_cost_and_usage.
32. Compare against a threshold (e.g., $50).
33. If exceeded, publish an SNS alert with the current spend.
34. Print the retrieved amount for logging.
35. EventBridge: Schedule daily.
36. Testing: Trigger manually with a low threshold (e.g., $0.01) to force an alert.
37. Discussion point: Mention AWS Budgets as the managed alternative and when custom Lambda logic wins (per-service breakdowns, Slack/Teams delivery, anomaly logic).

5.Restore an EC2 Instance from the Latest Snapshot

Objective: Automate disaster-recovery: rebuild an instance from its most recent EBS snapshot.
Instructions:

38. **Prerequisite: At least one snapshot of the source instance's root volume exists (Graded 2 pairs well here).**
39. **Lambda IAM Role: ec2:DescribeSnapshots, ec2:RegisterImage (or ec2:CreateImage), ec2:RunInstances, ec2:DescribeImages, ec2:CreateTags.**
40. **Lambda Function (Boto3):**
41. **Find the most recent snapshot for the given volume/instance (sort describe_snapshots by StartTime).**
42. **Register an AMI from the snapshot with register_image (specify root device mapping).**
43. **Launch a new t3.micro instance from that AMI and tag it (e.g., RestoredFrom=<snapshot-id>).**
44. **Print the new instance ID.**
45. **Testing: Trigger manually; verify the new instance boots and contains the snapshot's data. Terminate test instances afterwards to avoid charges.**

6.Audit S3 Buckets for Public Access and Notify

Objective: Detect any bucket that is publicly accessible and alert via SNS.
Note: Since April 2023, new buckets have Block Public Access enabled and ACLs disabled by default — so your audit must check both the Block Public Access configuration and bucket policy status, not just ACLs.
Instructions:

46. **SNS Setup: Topic + email subscription.**
47. **Lambda IAM Role: s3:ListAllMyBuckets, s3:GetBucketPublicAccessBlock, s3:GetBucketPolicyStatus, s3:GetBucketAcl, sns:Publish.**
48. **Lambda Function (Boto3):**
49. **List all buckets.**
50. **For each: check get_public_access_block, get_bucket_policy_status (IsPublic flag), and ACL grants.**
51. **If any bucket is public (or has Block Public Access disabled), publish an SNS alert naming it.**
52. **EventBridge: Schedule daily.**
53. **Testing: Deliberately disable Block Public Access on one test bucket and attach a public-read bucket policy; confirm the alert. Re-secure the bucket immediately after testing.**

From <https://vlearnv.herovired.com/page-activity/554/assign/138229>

Project 1: Automated S3 Bucket Cleanup

Sunday, July 12, 2026 4:55 PM

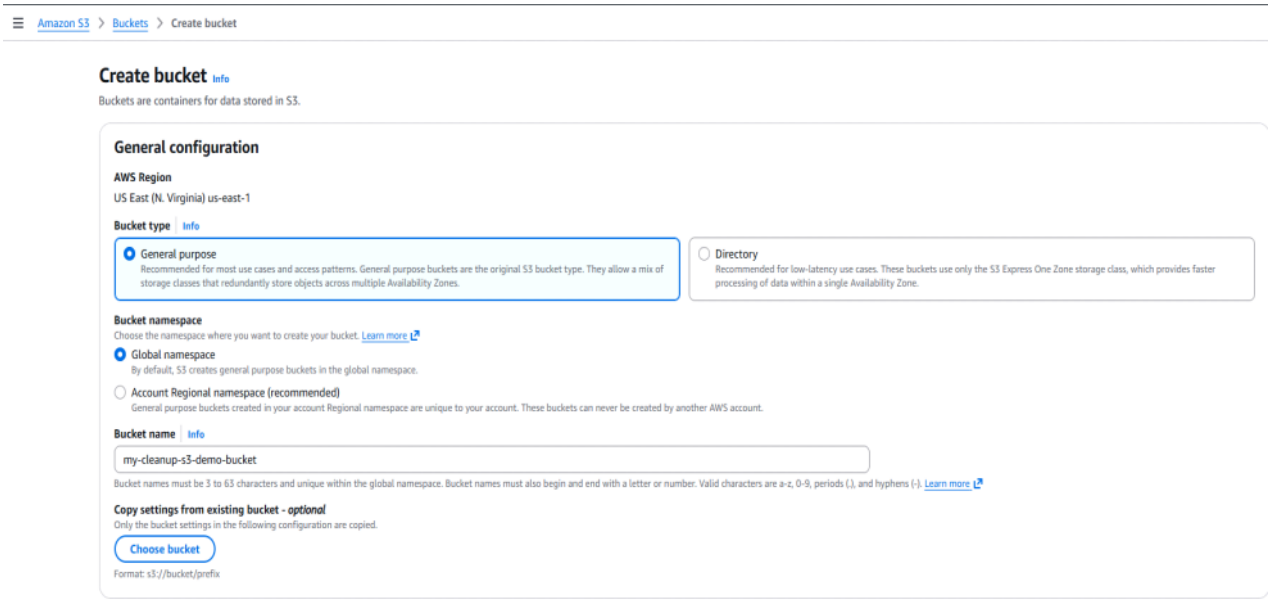
Task 1: Project 1: Automated S3 Bucket Cleanup (Objects Older Than 30 Days)

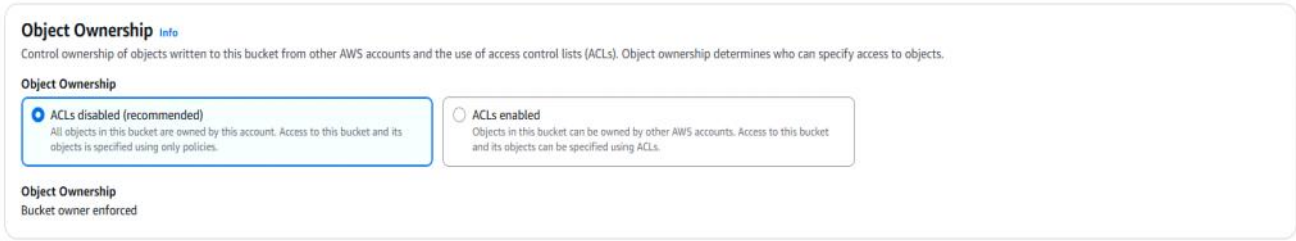
Step 1: S3 Bucket Setup

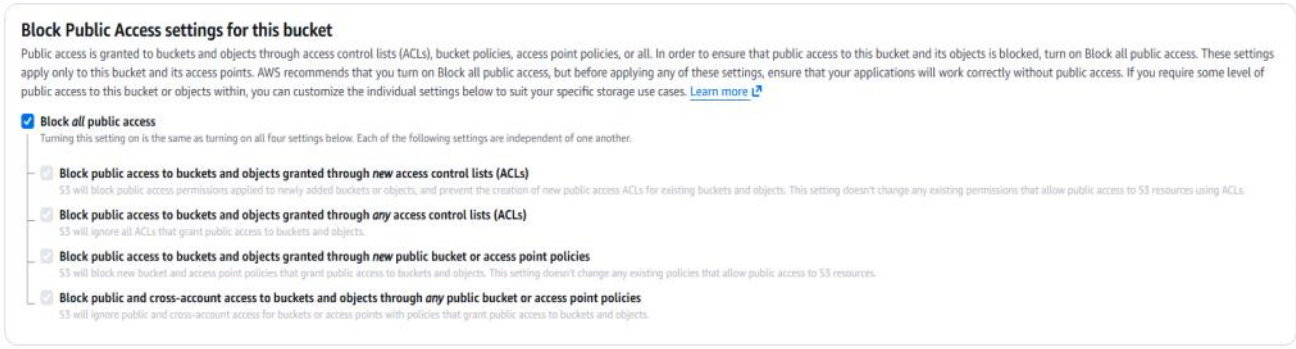
Option -1

AWS Console Navigation

1. Sign in to the **AWS Management Console**.
2. In the top search bar, type **S3** and open the **S3** service.
3. Click **Create bucket**.
4. **Bucket name:** my-cleanup-demo-bucket-12345 (must be globally unique — add your own suffix, e.g. initials or numbers).
5. **AWS Region:** choose your working region (e.g., us-east-1).
6. Leave **Block all public access** checked (default — keep it on).
7. Leave other settings default, scroll down, click **Create bucket**.
8. Open the bucket → click **Upload** → **Add files** → select 3–4 test files (e.g., file1.txt, file2.txt, file3.txt) → click **Upload**.







Option -2

CLI Equivalent:

aws s3 mb s3://my-cleanup-demo-bucket --region us-east-1

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> ls
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 mb s3://my-cleanup-demo-bucket --region us-east-1
make_bucket: my-cleanup-demo-bucket
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls
2026-07-13 13:51:19 my-cleanup-demo-bucket
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Create test files:
echo "test file 1" > file1.txt
echo "test file 2" > file2.txt
echo "test file 3" > file3.txt

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> echo "test file 1" > file1.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> echo "test file 2" > file2.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> echo "test file 3" > file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> ls -ltr
Get-ChildItem: A parameter cannot be found that matches parameter name 'ltr'.
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> ls

    Directory: D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---          7/13/2026  1:56 PM             13 file1.txt
-a---          7/13/2026  1:56 PM             13 file2.txt
-a---          7/13/2026  1:56 PM             13 file3.txt

PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Upload

aws s3 cp file1.txt s3://my-cleanup-demo-bucket/
aws s3 cp file2.txt s3://my-cleanup-demo-bucket/
aws s3 cp file3.txt s3://my-cleanup-demo-bucket/

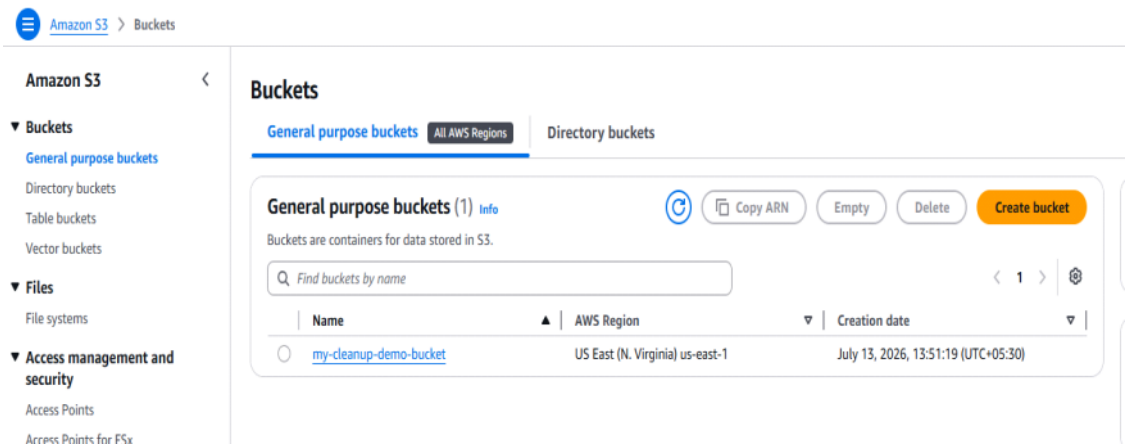
```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 cp file1.txt s3://my-cleanup-demo-bucket/
upload: .\file1.txt to s3://my-cleanup-demo-bucket/file1.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 cp file2.txt s3://my-cleanup-demo-bucket/
upload: .\file2.txt to s3://my-cleanup-demo-bucket/file2.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 cp file3.txt s3://my-cleanup-demo-bucket/
upload: .\file3.txt to s3://my-cleanup-demo-bucket/file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

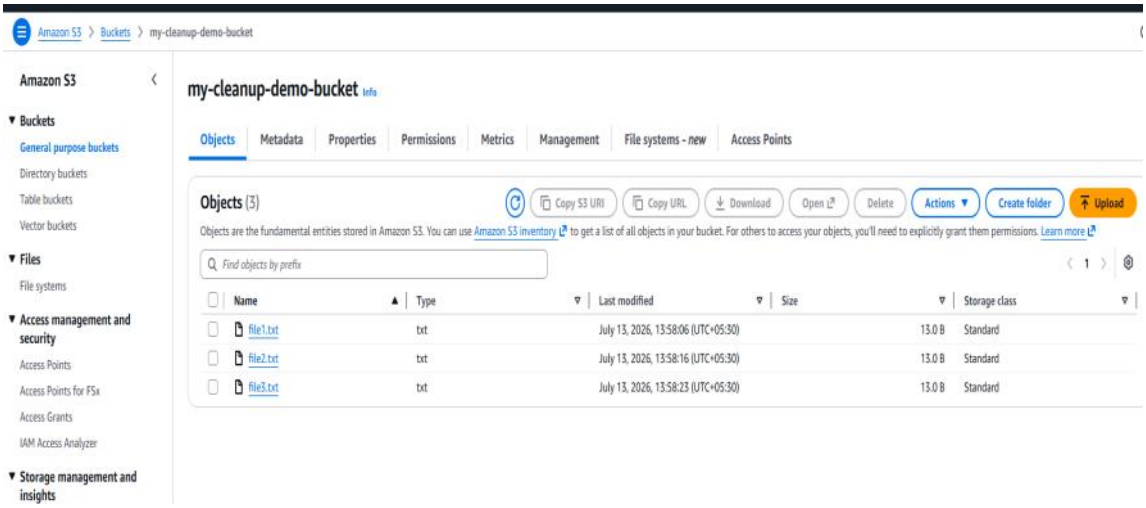
Verify

aws s3 ls s3://my-cleanup-demo-bucket/

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 13:58:06         13 file1.txt
2026-07-13 13:58:16         13 file2.txt
2026-07-13 13:58:23         13 file3.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

#Verify from UI:



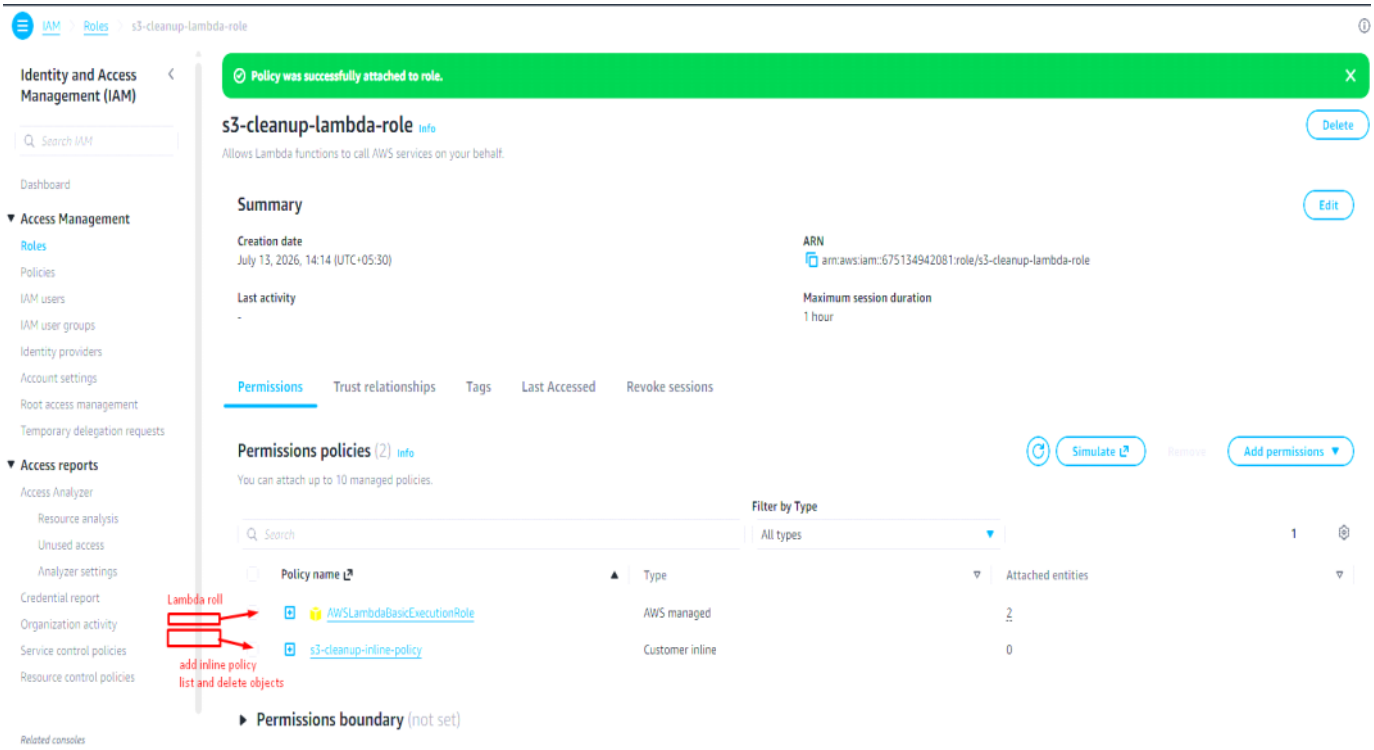


Note on testing "old" files: S3 sets Last Modified automatically at upload time — you cannot backdate it. So for testing, temporarily lower the age threshold to a few minutes (via a Lambda environment variable), upload files, wait past that window, run the function, and confirm deletion. Then switch the threshold back to 30 days for the "production" version.

Step 2: Lambda IAM Role

AWS Console Navigation

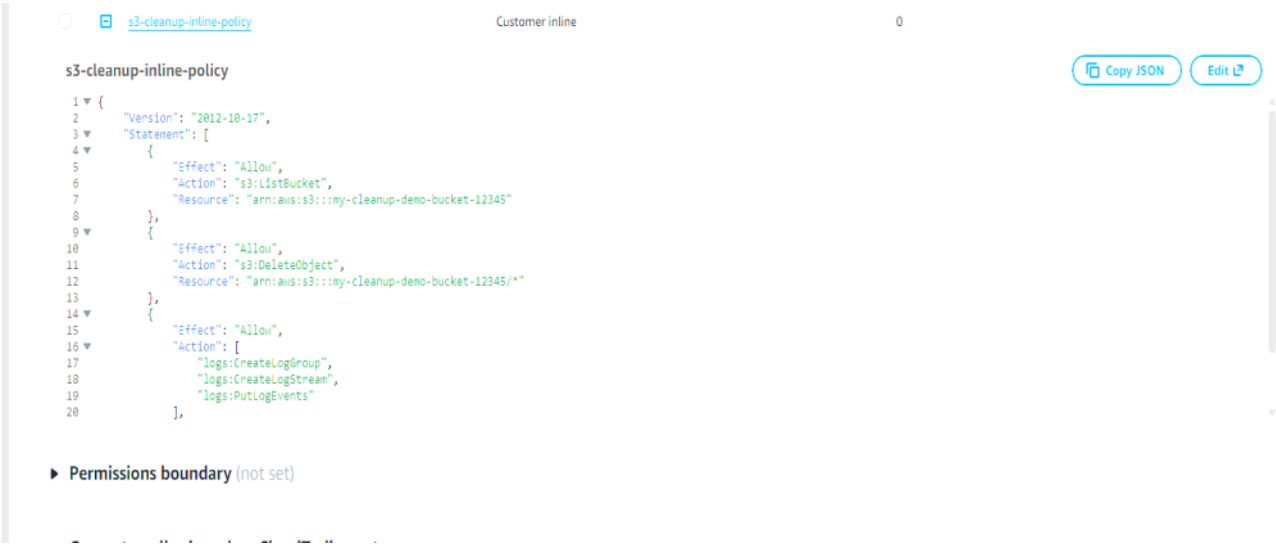
1. Search bar → type **IAM** → open **IAM** service.
2. Left sidebar → **Roles** → **Create role**.
3. **Trusted entity type:** AWS service.
4. **Use case:** select **Lambda** → click **Next**.
5. On the **Add permissions** page, skip attaching a managed policy for now (we'll add an inline policy after creation) → click **Next**.
6. **Role name:** s3-cleanup-lambda-role → click **Create role**.
7. Open the newly created role → **Add permissions** dropdown → **Create inline policy**.
8. Switch to the **JSON** tab and paste the policy below (see JSON block).
9. Click **Next**, name it s3-cleanup-inline-policy, click **Create policy**.
10. Also attach the AWS managed policy **AWSLambdaBasicExecutionRole** (Add permissions → Attach policies → search and select it) so the function can write to CloudWatch Logs.



s3-cleanup-inline-policy:

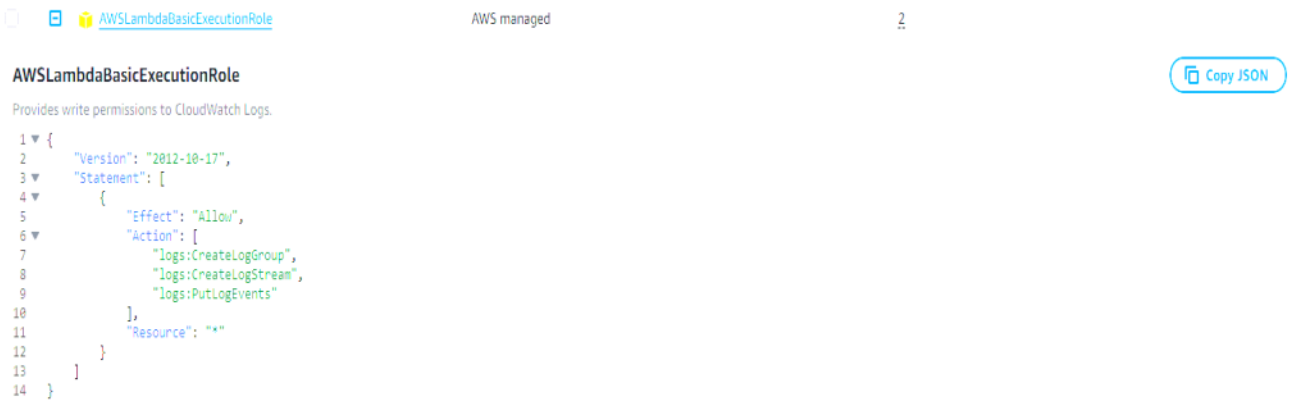
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::my-cleanup-demo-bucket"
    },
    {
      "Effect": "Allow",
      "Action": "s3:DeleteObject",
```

```
      "Resource": "arn:aws:s3:::my-cleanup-demo-bucket/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    }
  ]
}
```



AWSLambdaBasicExecutionRole:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```

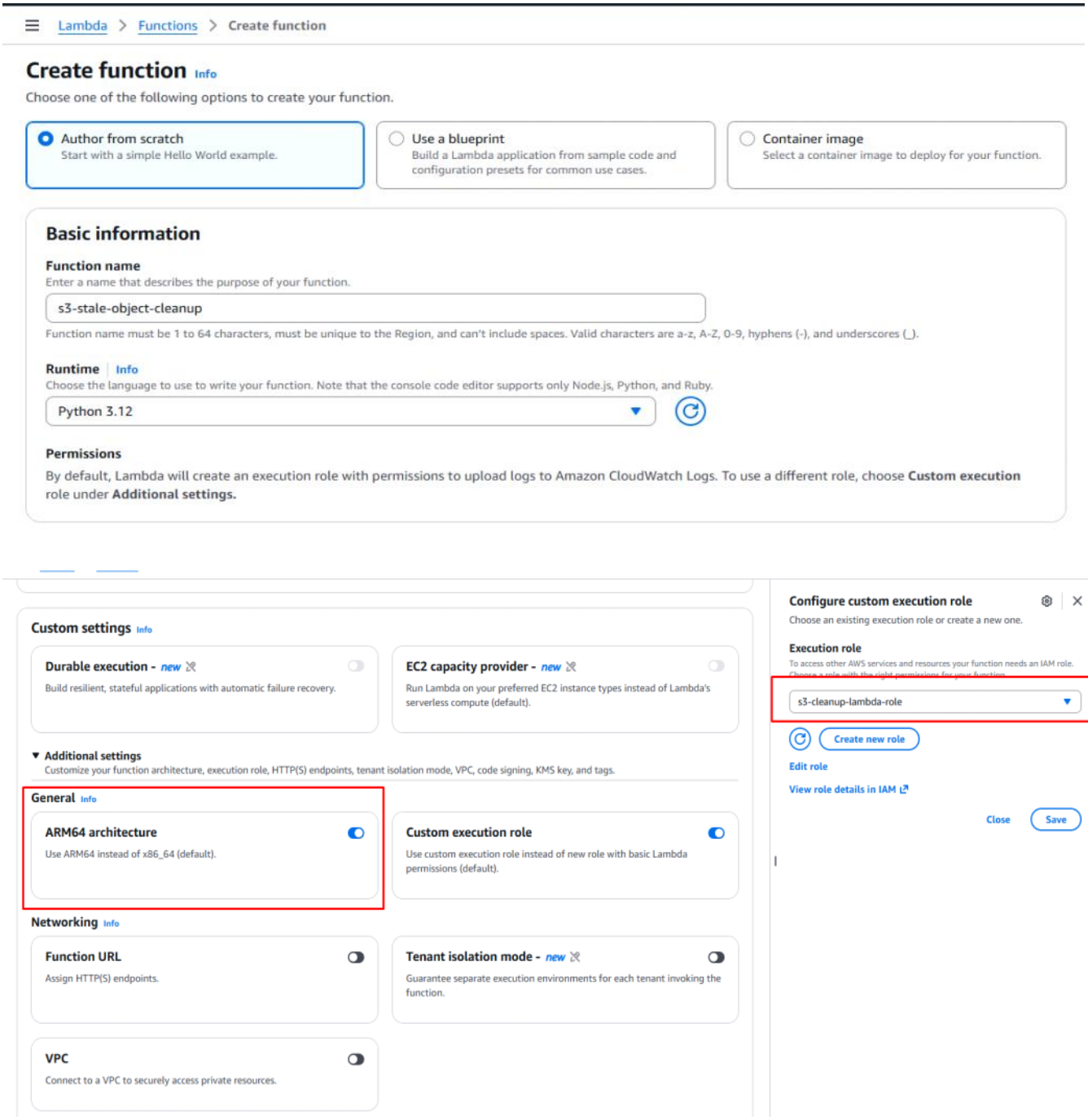


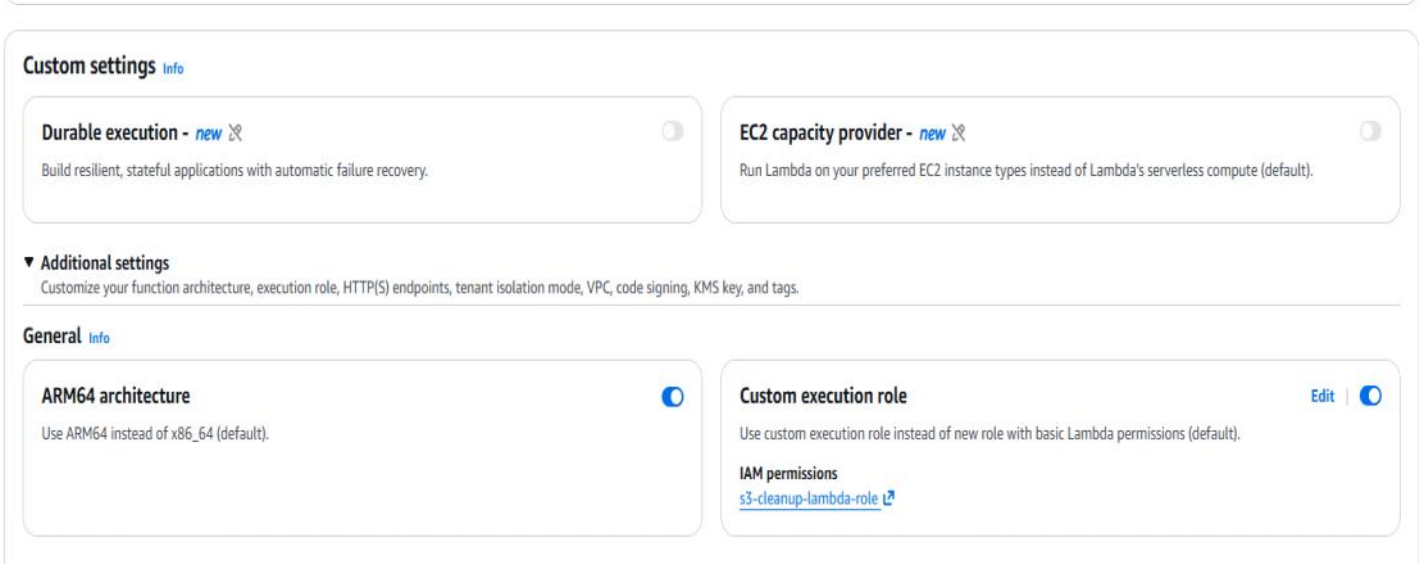
Important: s3:ListBucket applies to the **bucket ARN** (no trailing /*), while s3:DeleteObject applies to **object ARNs** (with /*). Mixing these up is the most common cause of AccessDenied errors.

Step 3: Lambda Function (Python 3.12, Boto3)

AWS Console Navigation

1. Search bar → type **Lambda** → open **Lambda** service.
2. Click **Create function**.
3. Choose **Author from scratch**.
4. **Function name:** s3-stale-object-cleanup.
5. **Runtime:** Python 3.12.
6. **Architecture:** x86_64 (default is fine).
7. Expand **Change default execution role** → select **Use an existing role** → choose s3-cleanup-lambda-role.
8. Click **Create function**.
9. In the **Code** tab, delete the placeholder code in lambda_function.py and paste the code below.
10. Click **Deploy** (top left of the code editor) to save.
11. Go to the **Configuration** tab → **Environment variables** → **Edit** → **Add environment variable**:
    - Key: BUCKET_NAME, Value: my-cleanup-demo-bucket-12345
    - Key: AGE_THRESHOLD_DAYS, Value: 30
    - Save.
12. Still under **Configuration** → **General configuration** → **Edit** → set **Timeout** to 1 min 0 sec (default 3 sec is too short) → **Save**.

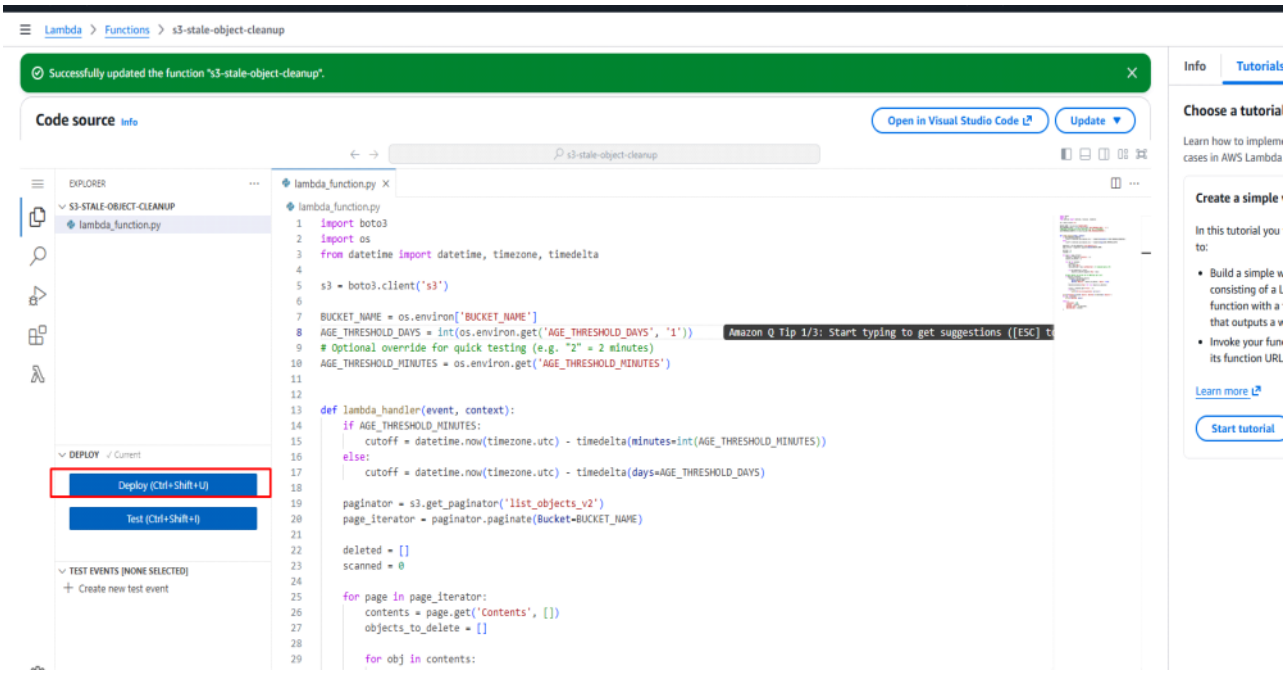

Lambda > Functions > Create function
Create function Info
Choose one of the following options to create your function.
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
s3-stale-object-cleanup
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.12
Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose Custom execution role under Additional settings.
Custom settings Info
Durable execution - new
Build resilient, stateful applications with automatic failure recovery.
EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.
General Info
ARM64 architecture
Use ARM64 instead of x86_64 (default).
Custom execution role
Use custom execution role instead of new role with basic Lambda permissions (default).
Networking Info
Function URL
Assign HTTP(S) endpoints.
Tenant isolation mode - new
Guarantee separate execution environments for each tenant invoking the function.
VPC
Connect to a VPC to securely access private resources.
Configure custom execution role
Choose an existing execution role or create a new one.
Execution role
s3-cleanup-lambda-role
Create new role
Edit role
View role details in IAM
Close
Save

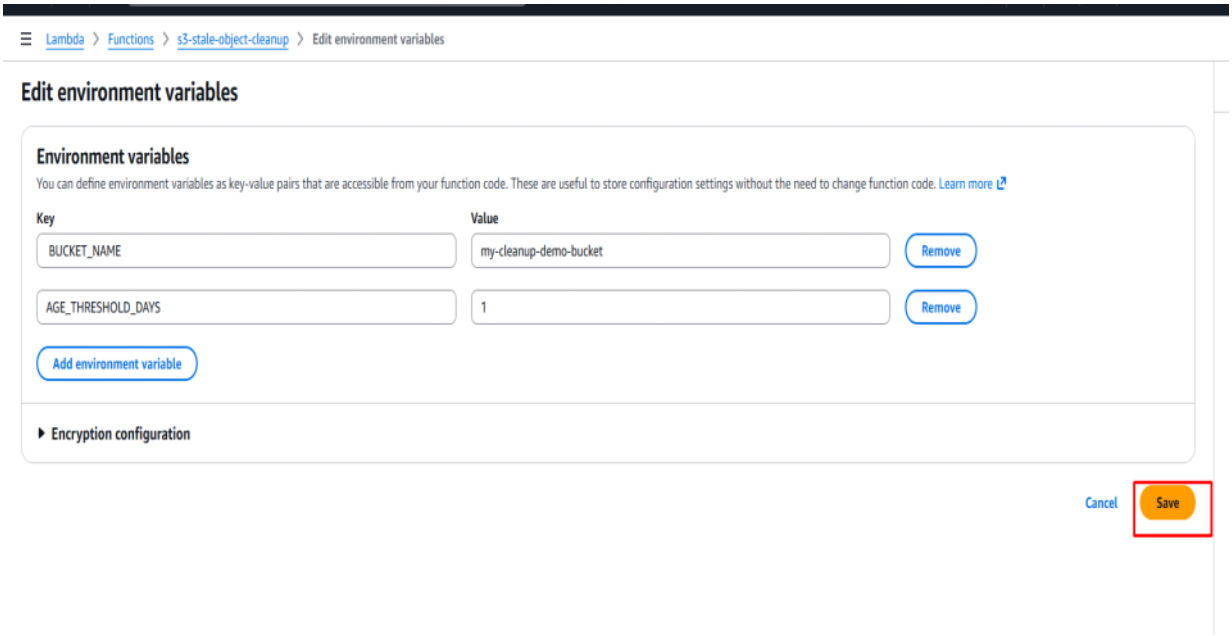


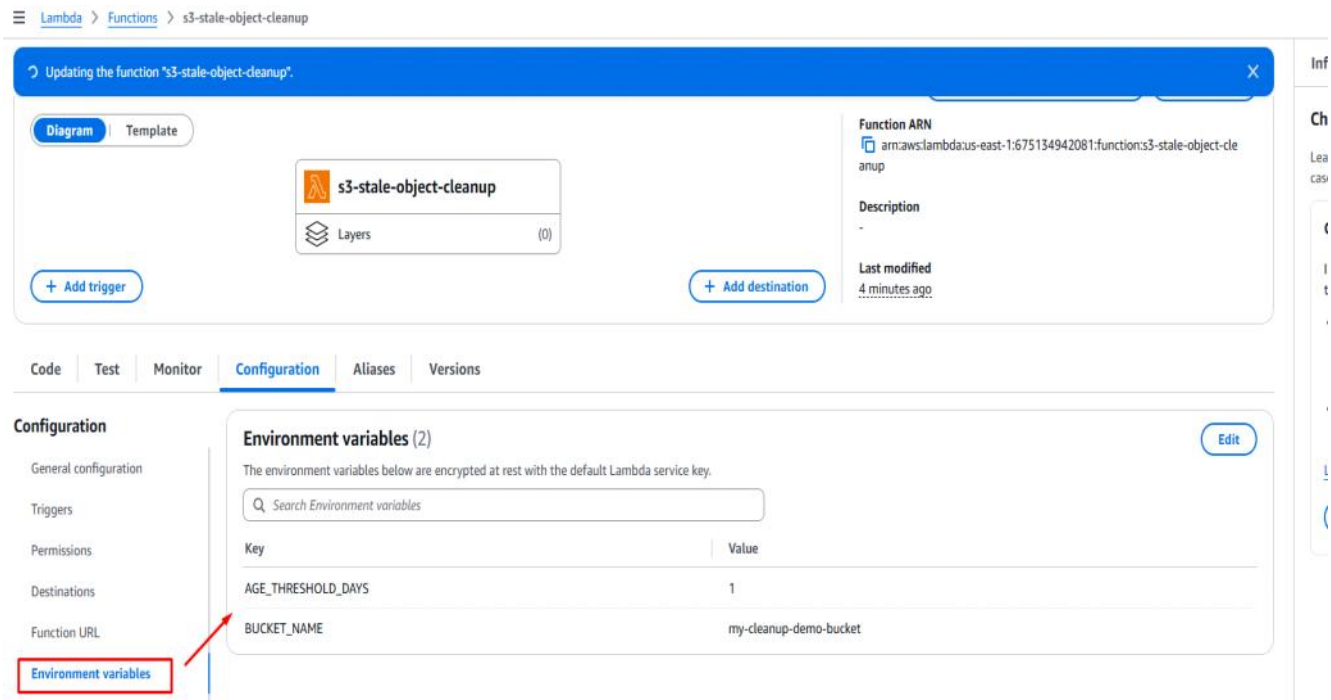
lambda_function.py:

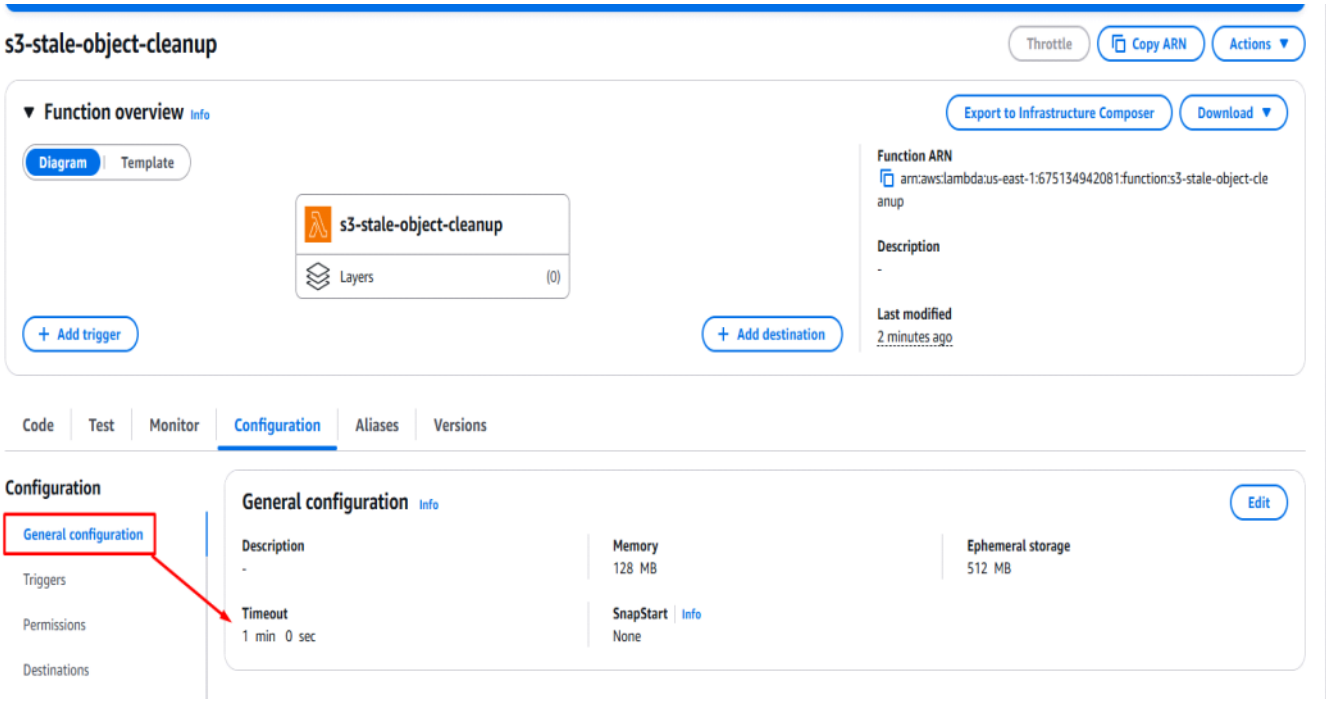
```
import boto3
import os
from datetime import datetime, timezone, timedelta

s3 = boto3.client('s3')

BUCKET_NAME = os.environ['BUCKET_NAME']
#AGE_THRESHOLD_DAYS = int(os.environ.get('AGE_THRESHOLD_DAYS', '1'))
# Optional override for quick testing (e.g. "2" = 2 minutes)
AGE_THRESHOLD_MINUTES = os.environ.get('AGE_THRESHOLD_MINUTES')


def lambda_handler(event, context):
    if AGE_THRESHOLD_MINUTES:
        cutoff = datetime.now(timezone.utc) - timedelta(minutes=int(AGE_THRESHOLD_MINUTES))
    else:
        cutoff = datetime.now(timezone.utc) - timedelta(days=AGE_THRESHOLD_DAYS)

    paginator = s3.get_paginator('list_objects_v2')
    page_iterator = paginator.paginate(Bucket=BUCKET_NAME)

    deleted = []
    scanned = 0

    for page in page_iterator:
        contents = page.get('Contents', [])
        objects_to_delete = []

        for obj in contents:
            scanned += 1
            key = obj['Key']
            last_modified = obj['LastModified']  # timezone-aware, UTC
```

```
            if last_modified < cutoff:
                objects_to_delete.append({'Key': key})

        # Batch delete (S3 allows up to 1000 keys per call)
        if objects_to_delete:
            response = s3.delete_objects(
                Bucket=BUCKET_NAME,
                Delete={'Objects': objects_to_delete, 'Quiet': True}
            )
            deleted.extend([o['Key'] for o in objects_to_delete])

            errors = response.get('Errors', [])
            if errors:
                print(f"Errors during delete: {errors}")

    print(f"Scanned {scanned} objects. Deleted {len(deleted)} objects.")
    for key in deleted:
        print(f"Deleted: {key}")

    return {
        'statusCode': 200,
        'scanned': scanned,
        'deleted_count': len(deleted),
        'deleted_keys': deleted
    }
```




Lambda > Functions > s3-stale-object-cleanup > Edit environment variables
Edit environment variables
Environment variables
You can define environment variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without the need to change function code. Learn more
Key
Value
BUCKET_NAME
my-cleanup-demo-bucket
Remove
AGE_THRESHOLD_DAYS
1
Remove
Add environment variable
Encryption configuration
Cancel
Save


Lambda > Functions > s3-stale-object-cleanup
Updating the function "s3-stale-object-cleanup".
Diagram
Template
s3-stale-object-cleanup
Layers
(0)
Add trigger
Add destination
Function ARN
arn:aws:lambda:us-east-1:675134942081:function:s3-stale-object-cleanup
Description
-
Last modified
4 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Environment variables (2)
Edit
The environment variables below are encrypted at rest with the default Lambda service key.
Search Environment variables
Key
Value
AGE_THRESHOLD_DAYS
1
BUCKET_NAME
my-cleanup-demo-bucket



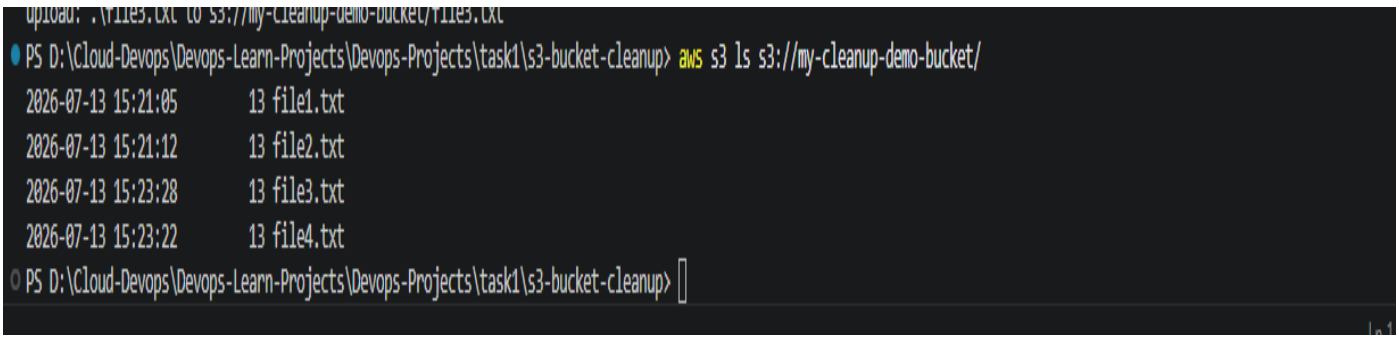
Step 4: Testing

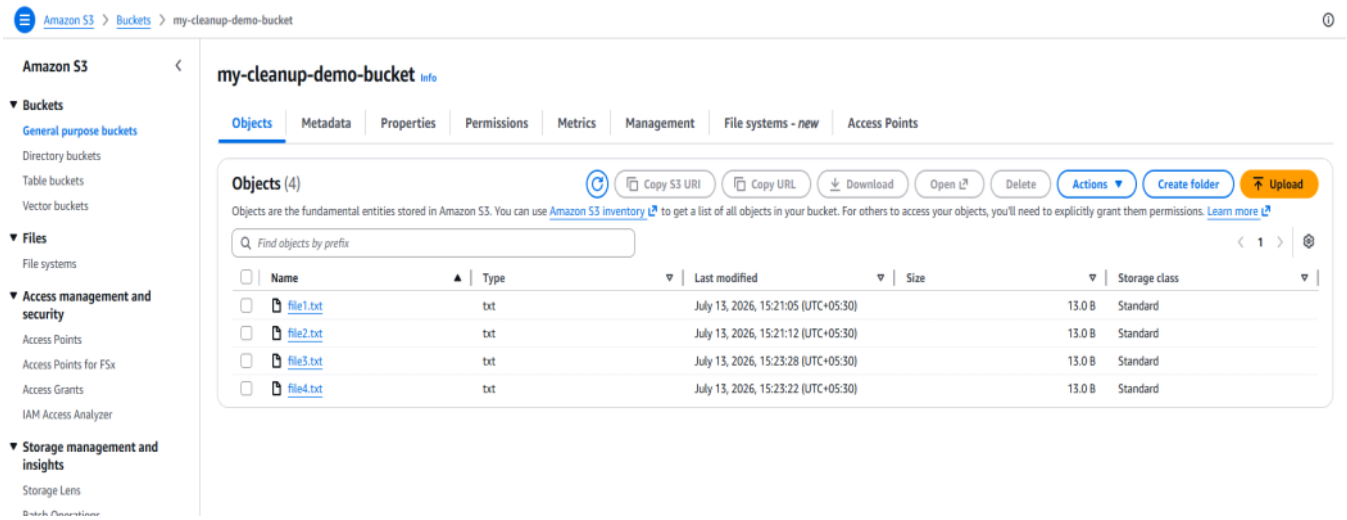
Phase A — Test with minutes threshold

Console:

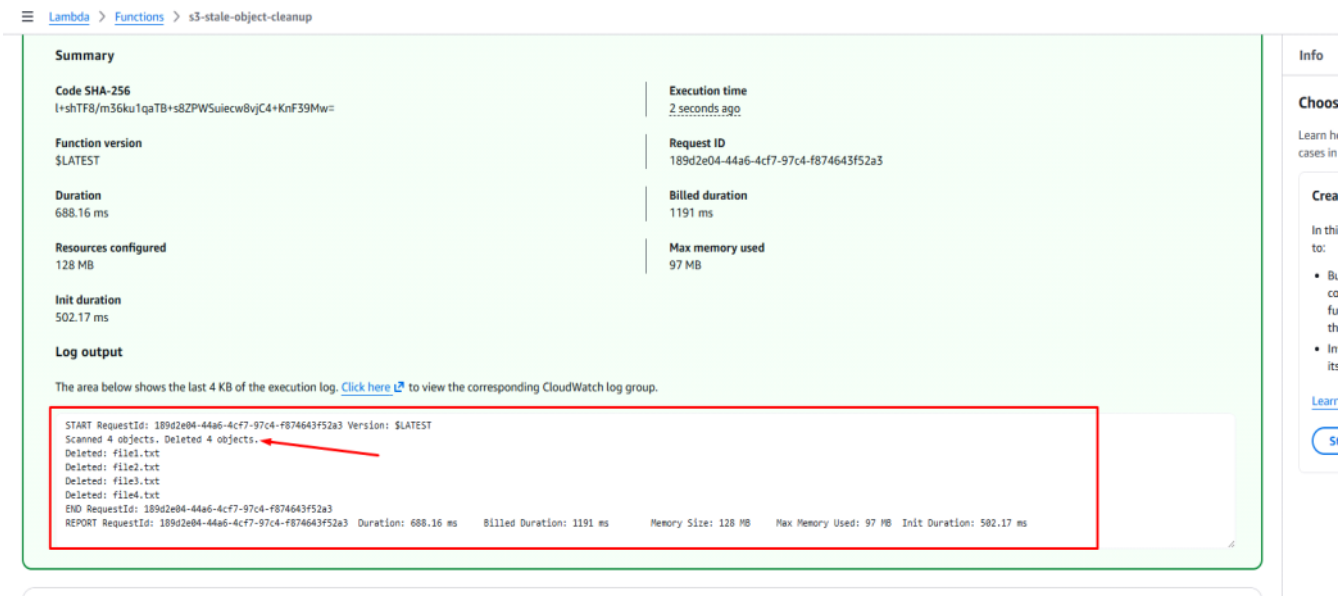
1. Go to Lambda function → **Configuration** → **Environment variables** → **Edit**.
2. Add a new variable: Key AGE_THRESHOLD_MINUTES, Value 2 → **Save**.
3. Upload a couple of test files to the bucket now (S3 console → Upload).
4. Wait 2–3 minutes.
5. Go to the **Test** tab → **Create new event** → Event name: manualTest → keep the default empty JSON {} (the function doesn't need event input) → **Save** → click **Test**.
6. Check the **Execution results** panel for output, and check **Monitor** tab → **View CloudWatch logs** to see the printed deleted key names.

Before run function:





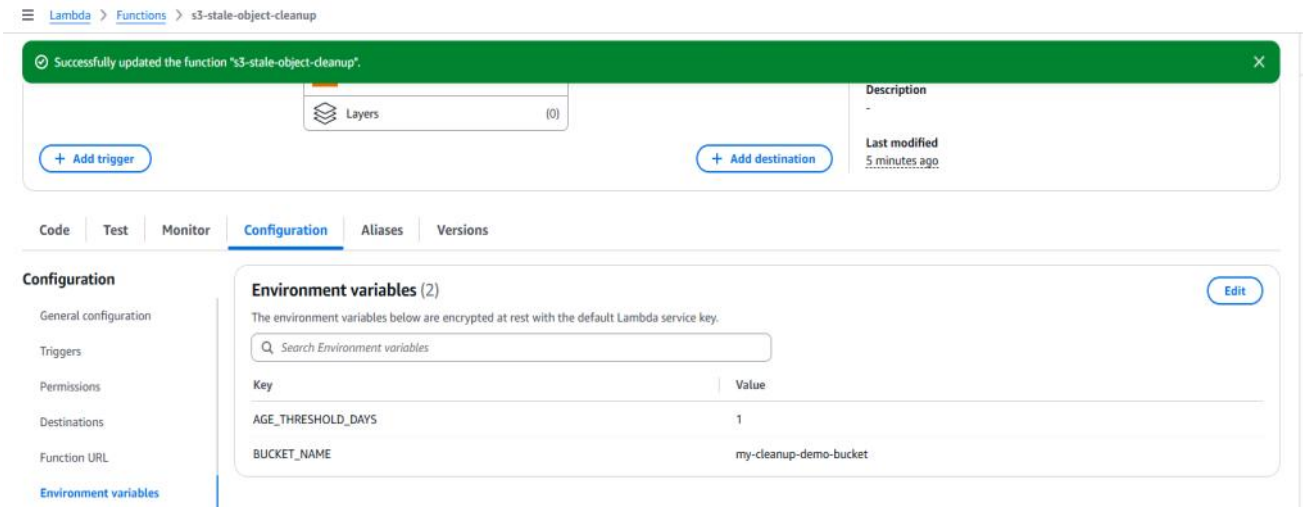
After manual run Lambda function:




```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:21:05         13 file1.txt
2026-07-13 15:21:12         13 file2.txt
2026-07-13 15:23:28         13 file3.txt
2026-07-13 15:23:22         13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Phase B — Reset to production threshold

Delete the AGE_THRESHOLD_MINUTES variable entirely **(leave AGE_THRESHOLD_DAYS=1)** → **Save**.



ReTest:

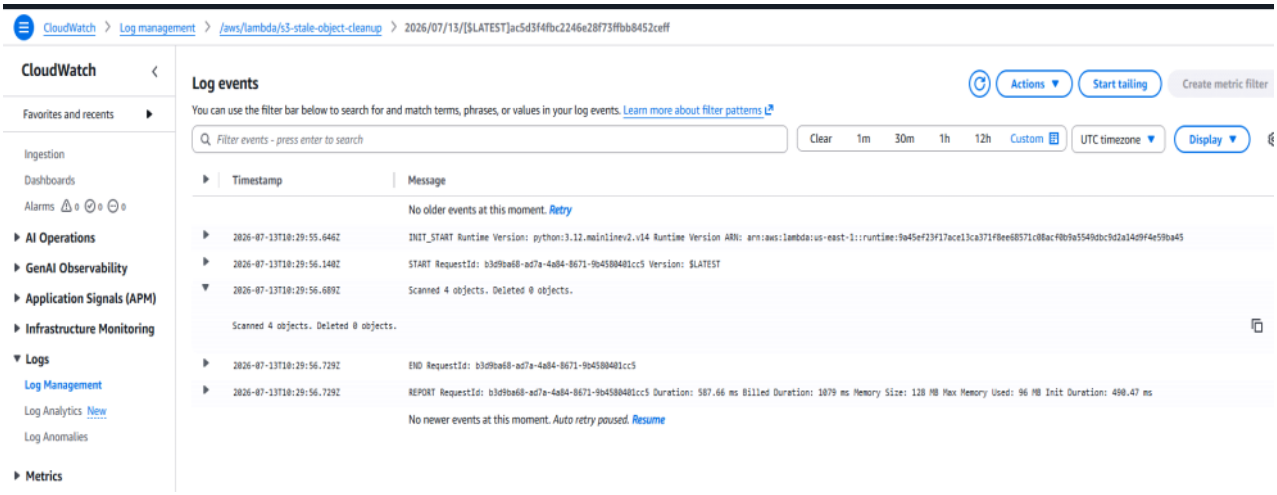
```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:56:25         13 file1.txt
2026-07-13 15:56:18         13 file2.txt
2026-07-13 15:56:07         13 file3.txt
2026-07-13 15:56:14         13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

After run Lambda function


```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
2026-07-13 15:56:25         13 file1.txt
2026-07-13 15:56:18         13 file2.txt
2026-07-13 15:56:07         13 file3.txt
2026-07-13 15:56:14         13 file4.txt
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```

Checking CloudWatch Logs (Console)

1. Search bar → **CloudWatch** → **Log groups**.
2. Find /aws/lambda/s3-stale-object-cleanup.
3. Open the latest **Log stream** to view scanned/deleted counts and the list of deleted keys.

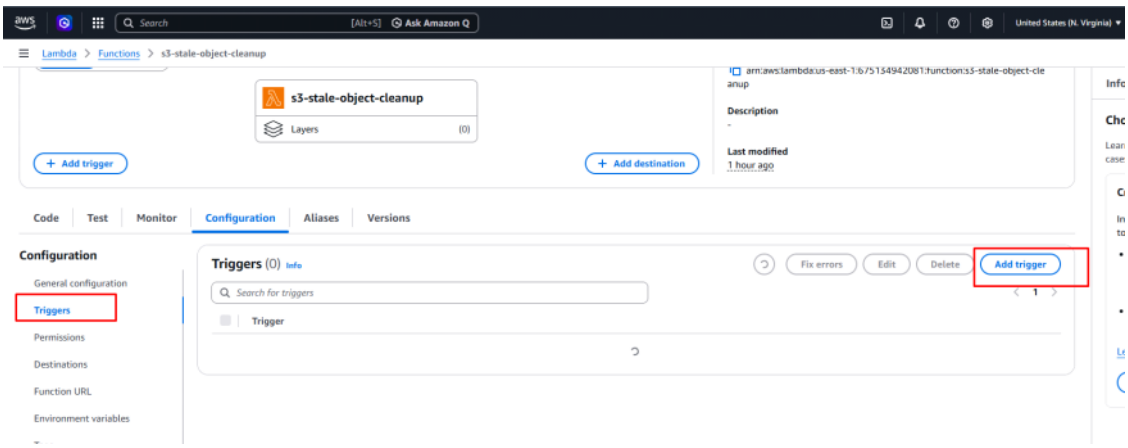


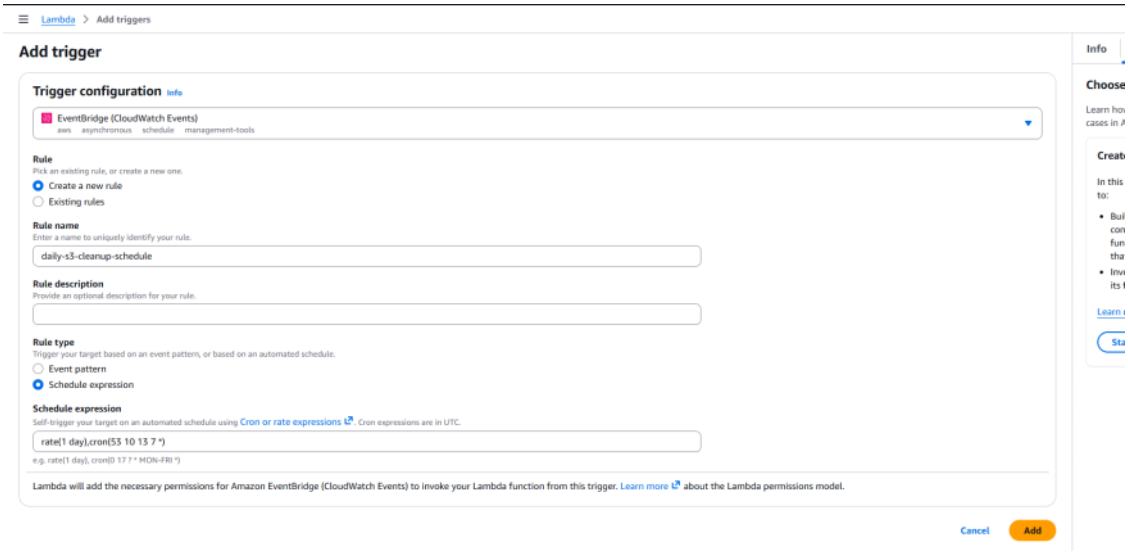
Step 5 (Optional): Schedule with EventBridge instead of manual trigger

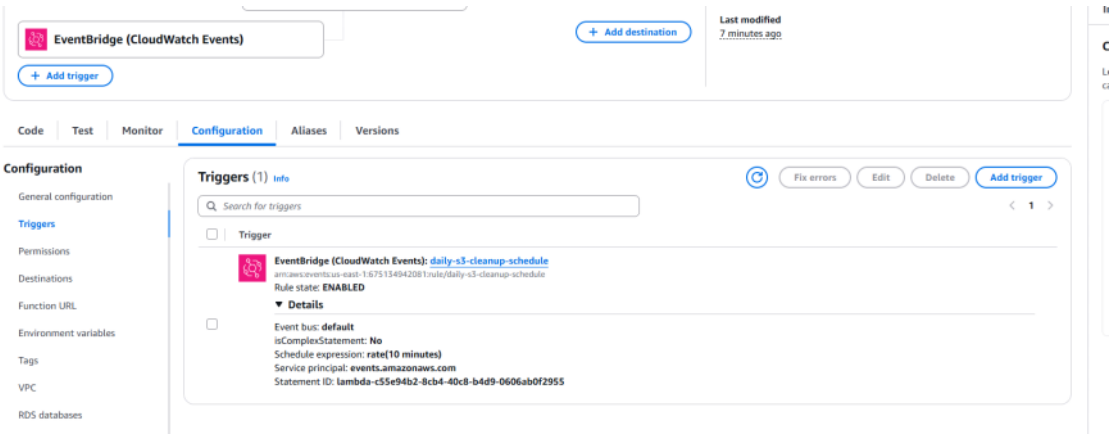
If you want this to run automatically (e.g., daily) rather than being manually invoked:

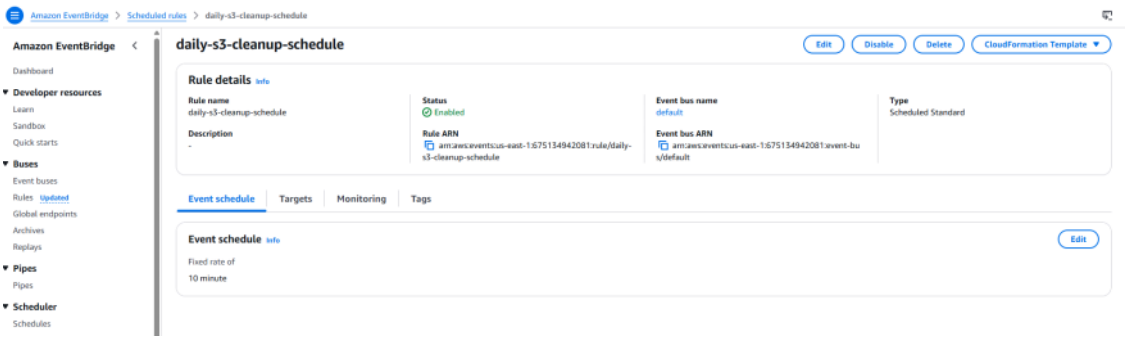
Console:

1. Go to the Lambda function → **Add trigger**.
2. Select **EventBridge (CloudWatch Events)**.
3. Choose **Create a new rule**.
4. Rule name: daily-s3-cleanup-schedule.
5. Rule type: **Schedule expression** → enter rate(1 day) (or a cron expression like cron(0 3 * * ? *) for 3 AM daily).
6. Click **Add**.



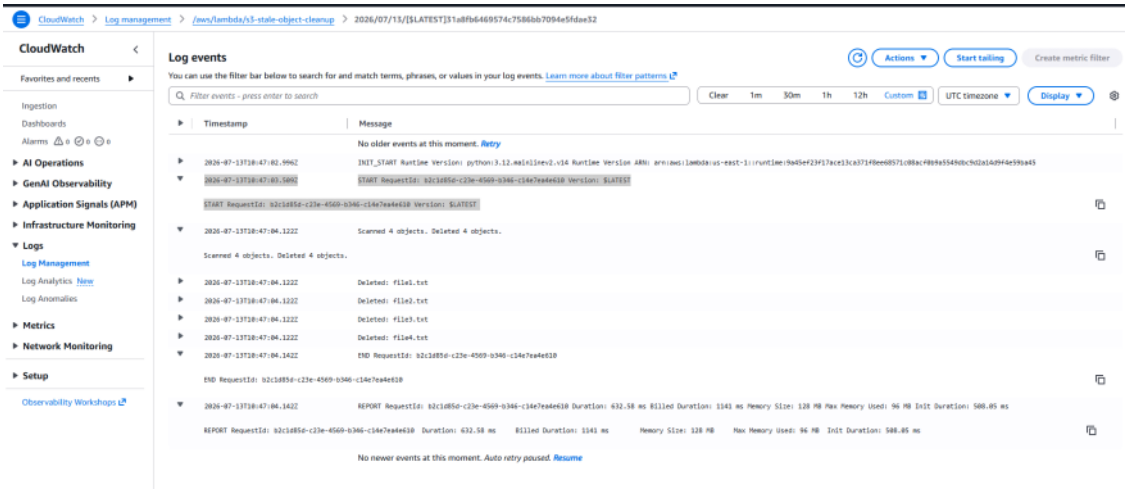






After 10 min execute :

```
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup> aws s3 ls s3://my-cleanup-demo-bucket/
PS D:\Cloud-Devops\Devops-Learn-Projects\Devops-Projects\task1\s3-bucket-cleanup>
```


CloudWatch > Log management > /aws/lambda/s3-stale-object-cleanup > 2026/07/13/[$LATEST]31a8fb6469574c7586bb7094e5fdae32
CloudWatch
Favorites and recents
Ingestion
Dashboards
Alarms
AI Operations
GenAI Observability
Application Signals (APM)
Infrastructure Monitoring
Logs
Log Management
Log Analytics New
Log Anomalies
Metrics
Network Monitoring
Setup
Observability Workshops
Log events
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Actions
Start tailing
Create metric filter
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
UTC timezone
Display
Timestamp
Message
No older events at this moment. Retry
2026-07-13T10:47:02.996Z
INIT_START Runtime Version: python:3.12.mai4i1ev2.v14 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:9a45ef23f17ace13ca37148ee68571c88ac480a55549dbc9d2a1a9f4e59ba45
2026-07-13T10:47:03.509Z
START RequestId: b2c1d85d-c23e-4569-b346-c14e7ea4e610 Version: $LATEST
START RequestId: b2c1d85d-c23e-4569-b346-c14e7ea4e610 Version: $LATEST
2026-07-13T10:47:04.122Z
Scanned 4 objects. Deleted 4 objects.
Scanned 4 objects. Deleted 4 objects.
2026-07-13T10:47:04.122Z
Deleted: file1.txt
2026-07-13T10:47:04.122Z
Deleted: file2.txt
2026-07-13T10:47:04.122Z
Deleted: file3.txt
2026-07-13T10:47:04.122Z
Deleted: file4.txt
2026-07-13T10:47:04.142Z
END RequestId: b2c1d85d-c23e-4569-b346-c14e7ea4e610
END RequestId: b2c1d85d-c23e-4569-b346-c14e7ea4e610
2026-07-13T10:47:04.142Z
REPORT RequestId: b2c1d85d-c23e-4569-b346-c14e7ea4e610 Duration: 632.58 ms Billed Duration: 1141 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 508.05 ms
REPORT RequestId: b2c1d85d-c23e-4569-b346-c14e7ea4e610 Duration: 632.58 ms Billed Duration: 1141 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 508.05 ms
No newer events at this moment. Auto retry paused. Resume